PRACTICAL - 06

Aim: Remote control (Switchable.java + Driver.java)

Code Screenshot:

Visual Studio Code - Switchable.java

Switchable.java

```java
public interface Switchable {
    void on();
    void off();
    boolean isOn();

    default void toggle() {
        if (isOn()) off();
        else on();
    }
}

class Fan implements Switchable {
    private boolean state;
    public void on() { state = true; System.out.println("Fan ON"); }
    public void off() { state = false; System.out.println("Fan OFF"); }
    public boolean isOn() { return state; }
}

class Light implements Switchable {
    private boolean state;
    public void on() { state = true; System.out.println("Light ON"); }
    public void off() { state = false; System.out.println("Light OFF"); }
    public boolean isOn() { return state; }
}

@FunctionalInterface
interface SwitchRule {
    boolean maySwitchOn(Switchable device, int hour);
}
```

Visual Studio Code - Driver.java

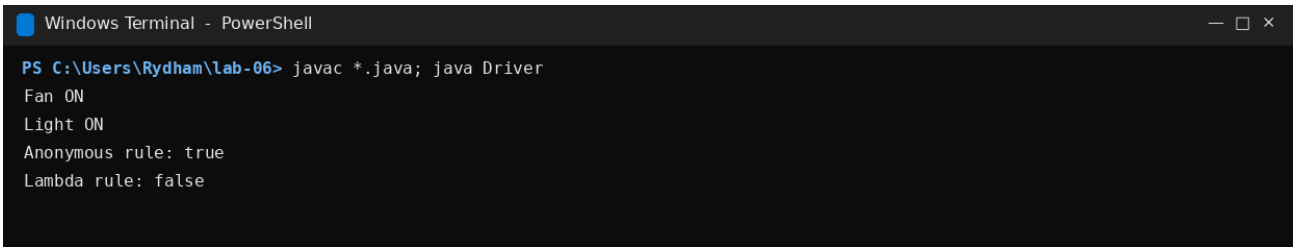
Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Switchable[] devices = {new Fan(), new Light()};
        for (Switchable d : devices) d.toggle();

        SwitchRule anonymous = new SwitchRule() {
            public boolean maySwitchOn(Switchable d, int hour) {
                return d != null && hour >= 6 && hour <= 22;
            }
        };
        SwitchRule lambda = (d, hour) -> d != null && hour >= 6 && hour <= 22;

        System.out.println("Anonymous rule: " + anonymous.maySwitchOn(devices[0], 10));
        System.out.println("Lambda rule: " + lambda.maySwitchOn(devices[1], 23));
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned interface definition, contract adherence, and default methods.
- Understood functional interfaces (@FunctionalInterface) in Java.
- Mastered the transition from anonymous inner classes to concise lambda expressions.
- Implemented multiple inheritance of type through interfaces.
- Understood default method evolution and method dispatch rules.

Aim: Notification senders (Notifier.java + Driver.java)

Code Screenshot:

Visual Studio Code - Notifier.java

Notifier.java

```java
@FunctionalInterface
public interface Notifier {
    void send(String message);
}

interface Urgent { }
```

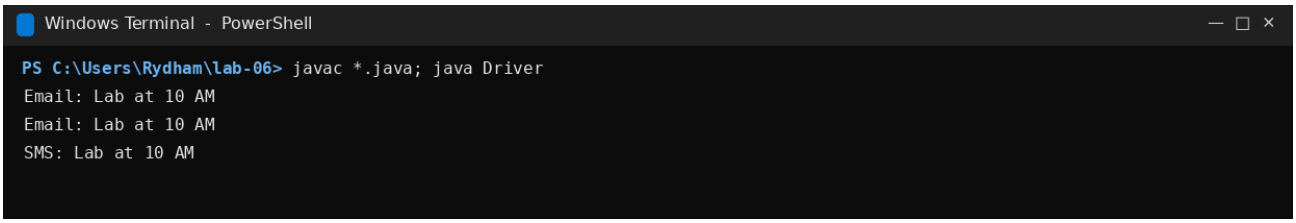
Visual Studio Code - Driver.java

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Notifier email = (Notifier & Urgent)
                msg -> System.out.println("Email: " + msg);
        Notifier sms = msg -> System.out.println("SMS: " + msg);
        Notifier[] senders = {email, sms};
        String message = "Lab at 10 AM";

        for (Notifier sender : senders) {
            sender.send(message);
            if (sender instanceof Urgent) sender.send(message);
        }
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered functional programming concepts in Java using lambda expressions.
- Understood marker interfaces and how they provide metadata at runtime.
- Learned how to treat behaviors as first-class citizens in arrays.
- Implemented broadcast notification patterns with conditional dispatch.
- Practiced clean separation between interface definitions and caller logic.

Aim: Discount engine (DiscountEngine.java + Driver.java)

Code Screenshot:

Visual Studio Code - DiscountEngine.java

DiscountEngine.java

```java
package discount;

import java.util.List;

public class DiscountEngine {
    @FunctionalInterface
    public interface Rule {
        double apply(double price);
    }

    public static void apply(List<Double> prices, Rule rule) {
        for (double price : prices)
            System.out.printf("%.0f -> %.0f%n", price, rule.apply(price));
    }
}
```

Visual Studio Code - Driver.java

Driver.java

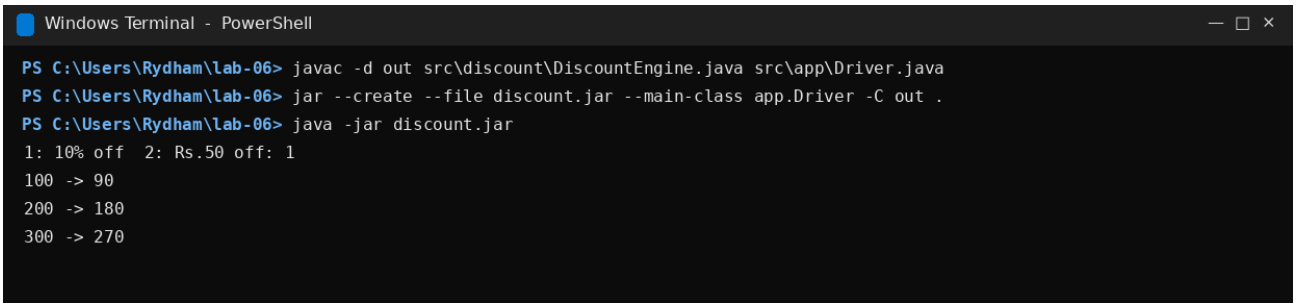
```java
package app;

import java.util.List;
import java.util.Scanner;
import discount.DiscountEngine;

public class Driver {
    public static void main(String[] args) {
        List<Double> prices = List.of(100.0, 200.0, 300.0);
        Scanner input = new Scanner(System.in);
        System.out.print("1: 10% off  2: Rs.50 off: ");
        int choice = input.nextInt();

        DiscountEngine.Rule rule = switch (choice) {
            case 1 -> price -> price * 0.90;
            case 2 -> price -> Math.max(0, price - 50);
            default -> price -> price;
        };
        DiscountEngine.apply(prices, rule);
        input.close();
    }
}
```

Output Screenshot:



Learning Outcome:

- Designed a functional discount rule using lambda expressions.
- Selected discount behaviour at runtime based on user input.
- Applied a common rule to multiple prices through a reusable engine.
- Organised Java classes into separate packages.
- Built and executed a runnable JAR using a manifest main class.